

DSA PATTERNS

FINAL STUDY REFERENCE • JAVA • INTERVIEW + LEETCODE

A pattern-first guide designed to answer the five questions you should ask before coding: **What pattern is this? Why does it work? What invariant am I maintaining? What template should I start from? What is the complexity?**

Layer	What you learn	Target
1	Pattern recognition	Identify the right family from wording/constraints.
2	Variants	Understand the different forms of the same pattern.
3	Invariant / state	Know exactly what must remain true while coding.
4	Template	Start from a reliable Java skeleton.
5	Practice ladder	Simple → Easy → Medium → LeetCode-level.
6	Failure modes	Know why common solutions break.

How to use this book

Do not memorize solutions. For every pattern: **learn recognition** → **understand invariant** → **write template without notes** → **solve 2–3 simple problems** → **solve medium problems** → **revisit mistakes** → **then timed LeetCode practice.**

00

Pattern Roadmap

The order to study patterns so each layer builds on the previous one

Stage	Patterns	Why here
01	Two Pointers	Foundation for ordered arrays, pairs and in-place transformations.
02	Sliding Window	Extends pointer movement to contiguous ranges.
03	Prefix Sum / Difference	Turns repeated range calculations into $O(1)$ queries.
04	Hashing	Fast membership, frequency, complement and grouping.
05	Binary Search	Search an ordered space, including answer-space search.
06	Sorting + Intervals	Normalize data, then reason about overlaps/order.
07	Stack / Monotonic Stack	Next/previous greater/smaller and boundary problems.
08	Queue / BFS	Layer-by-layer exploration and shortest unweighted paths.
09	Heap / Top-K	Repeated best candidate selection.
10	Linked List Pointers	Slow/fast, reversal and merge patterns.
11	Trees: DFS/BFS	Recursive structure, levels, paths and properties.
12	Backtracking	Explore choices with undo.
13	Greedy	Make locally optimal choices when an exchange/staying-ahead proof exists.
14	Graphs	Traversal, components, cycles, shortest paths and topological order.
15	Dynamic Programming	Overlapping subproblems + optimal substructure.
16	Advanced	Union-Find, Trie, Bit tricks, advanced monotonic structures.

Important

Patterns overlap. A single problem may combine **sorting + two pointers + greedy**, or **hashing + sliding window**. Your goal is to identify the controlling pattern, then add the supporting tool.

01

Pattern Recognition

Turn problem wording + constraints into a first hypothesis

The 10-second scan

Question	Look for
What is the input shape?	Array, string, linked list, tree, grid, graph.
Is the answer about a contiguous range?	Sliding window / prefix sum.
Is the data sorted or can I sort it?	Two pointers / binary search / intervals.
Do I need fast "seen/count/map"?	HashMap / HashSet.
Do I repeatedly need min/max?	PriorityQueue / monotonic structure.
Is there a previous/next relationship?	Stack / monotonic stack.
Is it layer-by-layer?	Queue / BFS.
Are there choices + undo?	Backtracking.
Are subproblems repeating?	Dynamic programming.
Can I prove a local choice is safe?	Greedy.

Constraint clues

Approximate n	Common target
$n \leq 20$	Exponential/backtracking may be possible.
n around 10^2	$O(n^3)$ can sometimes work.
n around 10^3	$O(n^2)$ often possible.
n around 10^4	Aim for $O(n \log n)$ or $O(n)$.
n around 10^5	Usually $O(n)$, sometimes $O(n \log n)$ depending on language/time.
Huge value range but small n	Compression / hashing / coordinate ideas may matter.

Never choose from keywords alone

"Maximum" does not automatically mean greedy; "sorted" does not automatically mean binary search. Combine **input structure + required output + constraints + operation pattern**.

02

Two Pointers

Use two indices to reduce repeated work, usually $O(n)$ after ordering

Core idea

Two pointers means maintaining two positions and moving them according to an invariant. The most common forms are **opposite ends**, **same direction**, and **fast/slow**.

Variant A — opposite ends

Recognition: sorted array + pair condition, palindrome, reverse/in-place partition. Move left/right based on whether the current state is too small or too large.

```
int left = 0, right = arr.length - 1;

while (left < right) {
    int sum = arr[left] + arr[right];

    if (sum == target) {
        // found
        break;
    } else if (sum < target) {
        left++;
    } else {
        right--;
    }
}
```

Why it works

For a sorted array, if $arr[left] + arr[right]$ is too small, keeping the same left and decreasing right cannot increase the sum, so **left must move**. Symmetrically, if the sum is too large, **right must move**.

Variant B — same direction / read-write

```
int write = 0;

for (int read = 0; read < arr.length; read++) {
    if (arr[read] != 0) {
        arr[write++] = arr[read];
    }
}
// optionally fill arr[write..] with zeros
```

Use when one pointer scans input and another marks where the next accepted element should go. Common for removing duplicates, moving zeros and partition-like transformations.

Variant C — fast/slow

One pointer moves faster than another. Common in linked lists for middle/cycle detection, but also useful for arrays when the slow pointer tracks a valid prefix.

```
Node slow = head;
Node fast = head;

while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
}
```

Two-pointer checklist

- Can I eliminate one candidate every time I move a pointer?
- Is the array sorted, or can sorting preserve the required answer?
- What invariant describes the region I have already processed?
- If duplicates exist, do I need to skip equal values?

Classic progression

Start with pair sum → palindrome → remove duplicates → move zeros → 3Sum → container/area-style problems → linked-list fast/slow.

03

Sliding Window

Maintain a contiguous range while expanding and shrinking

Recognition

Use sliding window when the answer concerns a **contiguous subarray or substring** and you can update the window state incrementally.

Variable-size window template

```
int left = 0;
int answer = 0;

for (int right = 0; right < n; right++) {
    // add arr[right] to window state

    while (/* window invalid */) {
        // remove arr[left] from window state
        left++;
    }

    answer = Math.max(answer, right - left + 1);
}
```

Fixed-size window

```
long sum = 0;

for (int i = 0; i < n; i++) {
    sum += arr[i];

    if (i >= k) {
        sum -= arr[i - k];
    }

    if (i >= k - 1) {
        // window [i-k+1, i]
    }
}
```

Window types

Type	State	Typical question
Fixed size	size exactly k	Maximum sum of every k elements.
At most K	condition breaks when count > K	Longest subarray with $\leq K$ distinct.
Exactly K	often solve $\text{atMost}(K) - \text{atMost}(K-1)$	Exactly K distinct.
Minimum valid	shrink while valid; update minimum	Minimum window meeting target.
Maximum valid	expand; shrink only when invalid	Longest valid substring.

Invariant

At every point, the current $[\text{left}, \text{right}]$ window satisfies your chosen validity condition after the shrink loop. This is the heart of the proof.

Common trap

Do not blindly use sliding window for arbitrary subarrays. It works best when the window can be updated locally and the validity/score behaves predictably as pointers move.

04

Prefix Sum & Difference Array

Precompute cumulative information to answer range operations quickly

Prefix sum

`prefix[i]` stores the sum of elements before/through a chosen boundary. The standard form uses prefix length $n+1$ so range arithmetic becomes clean.

```
long[] prefix = new long[n + 1];

for (int i = 0; i < n; i++) {
    prefix[i + 1] = prefix[i] + arr[i];
}

// sum of arr[l..r] inclusive:
long sum = prefix[r + 1] - prefix[l];
```

When to use

- Many range-sum queries on a static array.
- Subarray sum reasoning.
- Turning a repeated $O(n)$ range calculation into $O(1)$ per query after $O(n)$ preprocessing.

Prefix + HashMap: subarray sum

```
Map<Long, Integer> first = new HashMap<>();
first.put(0L, -1);

long sum = 0;
for (int i = 0; i < n; i++) {
    sum += arr[i];

    if (first.containsKey(sum - target)) {
        // a previous prefix makes a target-sum subarray
    }

    first.putIfAbsent(sum, i);
}
```

The key equation is **`prefix[j] - prefix[i] = target`**, so if `prefix[j]` is known, we need an earlier prefix equal to `prefix[j] - target`.

Difference array

A difference array records changes at boundaries so a large number of range additions can be applied efficiently; a final prefix accumulation reconstructs the values.

```
int[] diff = new int[n + 1];

// add value x to [l, r]:
diff[l] += x;
diff[r + 1] -= x;

// reconstruct
int running = 0;
for (int i = 0; i < n; i++) {
    running += diff[i];
    arr[i] += running;
}
```

05

Hashing

Trade extra memory for fast average lookup

Recognition

Think HashMap/HashSet when the problem repeatedly asks: **Have I seen this? How many? Where was it? What complements it? What group does it belong to?**

Frequency

```
Map<Character, Integer> freq = new HashMap<>();

for (char c : s.toCharArray()) {
    freq.put(c, freq.getOrDefault(c, 0) + 1);
}
```

Complement pattern

```
Map<Integer, Integer> seen = new HashMap<>();

for (int i = 0; i < nums.length; i++) {
    int need = target - nums[i];

    if (seen.containsKey(need)) {
        // answer uses seen.get(need) and i
    }

    seen.put(nums[i], i);
}
```

First/last occurrence

Store an index when you need distance or earliest/latest position. Choose putIfAbsent when the **first** occurrence must survive.

Hashing variants

Need	Structure/state
Existence	HashSet
Frequency	HashMap
Index	HashMap
Grouping	HashMap
Prefix state	HashMap

Complexity truth

HashMap/HashSet operations are typically $O(1)$ average. Your algorithm may still be $O(n \log n)$ or worse because of sorting or other work around the hash operations.

06

Binary Search

Halve an ordered search space

Classic exact search

```
int left = 0, right = nums.length - 1;

while (left <= right) {
    int mid = left + (right - left) / 2;

    if (nums[mid] == target) return mid;
    if (nums[mid] < target) left = mid + 1;
    else right = mid - 1;
}
return -1;
```

Boundary search — lower bound

Lower bound is the first index whose value is **at least** target. This form is more useful in DSA than exact search because it finds insertion points and boundaries.

```
int left = 0, right = nums.length;

while (left < right) {
    int mid = left + (right - left) / 2;

    if (nums[mid] >= target) right = mid;
    else left = mid + 1;
}
// left is first index with nums[index] >= target
```

Upper bound

Upper bound is the first index whose value is **greater than** target. It is useful for counting values $\leq$ target or locating the end of an equal range.

Binary search on answer

Instead of searching an array, search a numeric answer range. The key is a monotonic predicate: once feasible becomes true, it stays true (or vice versa).

```
long lo = 0, hi = 1_000_000_000L;

while (lo < hi) {
    long mid = lo + (hi - lo) / 2;

    if (feasible(mid)) hi = mid;
    else lo = mid + 1;
}
// lo is the minimum feasible answer
```

Recognition

- Sorted/monotonic data.
- Need first/last valid position.
- Can phrase the question as “is x feasible?” and feasibility changes only once.
- Need $O(\log n)$ search over a huge numeric answer space.

07

Sorting & Intervals

Sort first when order exposes the structure

Why sort?

Sorting often changes an unstructured problem into a linear scan because neighboring or adjacent items become meaningful. Typical cost is $O(n \log n)$.

Intervals — merge overlap

```
Arrays.sort(intervals, Comparator.comparingInt(a -> a[0]));

List<int[]> merged = new ArrayList<>();

for (int[] cur : intervals) {
    if (merged.isEmpty() ||
        merged.get(merged.size() - 1)[1] < cur[0]) {
        merged.add(new int[]{cur[0], cur[1]});
    } else {
        int[] last = merged.get(merged.size() - 1);
        last[1] = Math.max(last[1], cur[1]);
    }
}
```

Interval variants

Problem shape	Sort by	Then...
Merge intervals	start	Extend current end.
Meeting overlap	start	Track active end / use heap.
Meeting rooms	start or event times	Use min-heap of ending times or event sweep.
Insert interval	start	Merge before/overlap/after sections.
Activity selection	end	Greedily choose earliest finishing compatible interval.

Inclusive vs exclusive

Clarify whether $[1,3]$ and $[3,5]$ overlap. Many interval bugs come from not deciding endpoint semantics before coding.

Ordinary stack

Stack is useful when the problem naturally processes the most recently added unresolved item first: parentheses, undo-like state, expression parsing and nested structures.

Monotonic stack idea

Maintain a stack whose values/indices are monotonic. When a new element violates monotonicity, pop elements whose answer has just been discovered.

Next greater element

```
int[] ans = new int[n];
Arrays.fill(ans, -1);

Deque<Integer> stack = new ArrayDeque<>(); // indices

for (int i = 0; i < n; i++) {
    while (!stack.isEmpty() && nums[stack.peek()] < nums[i]) {
        ans[stack.pop()] = nums[i];
    }
    stack.push(i);
}
```

What to store?

Store	Why
Values	When only the value matters.
Indices	When you need distance, position, width or to access the original value.
Pairs/state	When each unresolved item needs extra information.

Monotonic direction

For next greater, a common pattern is a decreasing stack of unresolved values. For next smaller, reverse the comparison. The exact direction depends on the question.

Recognition

- "next/previous greater/smaller".
- Nearest element satisfying a comparison.
- Histogram / largest rectangle boundaries.
- Temperatures / stock span style problems.
- Need to resolve many elements once rather than compare every pair.

Complexity

Although there is a nested while-loop, each element is pushed once and popped at most once, giving $O(n)$ total time.

09

Queue & BFS

Explore states level by level

BFS template

```
Queue<Integer> q = new ArrayDeque<>();
boolean[] visited = new boolean[n];

q.offer(start);
visited[start] = true;

while (!q.isEmpty()) {
    int node = q.poll();

    for (int next : graph[node]) {
        if (!visited[next]) {
            visited[next] = true;
            q.offer(next);
        }
    }
}
```

Level-order BFS

```
q.offer(start);
int distance = 0;

while (!q.isEmpty()) {
    int size = q.size();

    for (int i = 0; i < size; i++) {
        int node = q.poll();
        // process this level

        // add next-level states
    }

    distance++;
}
```

Grid BFS

For an unweighted grid/graph, BFS finds the minimum number of edges/steps from the source when every move has equal cost.

```
int[] dr = {-1, 1, 0, 0};
int[] dc = {0, 0, -1, 1};

Queue<int[]> q = new ArrayDeque<>();
q.offer(new int[]{sr, sc});

while (!q.isEmpty()) {
    int[] cur = q.poll();

    for (int k = 0; k < 4; k++) {
        int nr = cur[0] + dr[k];
        int nc = cur[1] + dc[k];
        // bounds + blocked + visited checks
    }
}
```

BFS vs DFS

For **shortest path in an unweighted graph**, BFS is the standard first choice. DFS is for exploration/reachability and recursive structural reasoning, not shortest distance by default.

10

Heap / PriorityQueue

Keep the best candidate accessible

Recognition

Use a heap when you repeatedly need the smallest/largest element while new candidates keep arriving, or when you need Top-K.

Top-K pattern

```
PriorityQueue<Integer> minHeap = new PriorityQueue<>();

for (int x : nums) {
    minHeap.offer(x);

    if (minHeap.size() > k) {
        minHeap.poll();
    }
}
// heap contains k largest values
```

Why a min-heap for k largest? The smallest among the current k largest sits at the top, so it is the first candidate to remove when a better value arrives.

K smallest

Reverse the heap logic: use a max-heap of size k, so the largest among the retained k smallest values is easy to remove.

K-way merge

Put the first element from each sorted source into a min-heap. Pop the smallest and insert the next element from that same source. Complexity is typically $O(N \log K)$.

Heap checklist

- What is the priority?
- Can I keep only K candidates?
- Do I need the exact order or only repeated min/max access?
- Can each item be inserted/pollled only once?

11

Linked List Pointer Patterns

Manipulate references rather than indices

Reverse a list

```
Node prev = null;
Node cur = head;

while (cur != null) {
    Node next = cur.next;
    cur.next = prev;
    prev = cur;
    cur = next;
}

head = prev;
```

Find middle

```
Node slow = head, fast = head;

while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
}
```

Detect cycle

Floyd's algorithm moves slow by one and fast by two. If they ever meet, a cycle exists.

```
Node slow = head, fast = head;

while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;

    if (slow == fast) return true;
}

return false;
```

Merge two sorted lists

```
Node dummy = new Node(0);
Node tail = dummy;

while (a != null && b != null) {
    if (a.val <= b.val) {
        tail.next = a;
        a = a.next;
    } else {
        tail.next = b;
        b = b.next;
    }
    tail = tail.next;
}

tail.next = (a != null) ? a : b;
return dummy.next;
```

Reference safety

Before changing `cur.next`, save the old next node. Otherwise you can lose the rest of the list.

12

Tree DFS & BFS

Recursion for structure; queues for levels

DFS traversals

```
void preorder(Node root) {
    if (root == null) return;
    // root
    preorder(root.left);
    preorder(root.right);
}

void inorder(Node root) {
    if (root == null) return;
    inorder(root.left);
    // root
    inorder(root.right);
}

void postorder(Node root) {
    if (root == null) return;
    postorder(root.left);
    postorder(root.right);
    // root
}
```

What traversal means

Traversal	Order	Common use
Preorder	root → left → right	Serialize/copy tree; process parent before children.
Inorder	left → root → right	BST gives sorted order.
Postorder	left → right → root	Compute child-dependent values; delete/free-style reasoning.
Level order	level by level	Minimum depth, level widths, nearest nodes.

Tree DFS state

Many tree problems become easy when each recursive call returns a summary to its parent: height, sum, balance status, best path, etc.

```
int height(Node root) {
    if (root == null) return 0;

    int left = height(root.left);
    int right = height(root.right);

    return 1 + Math.max(left, right);
}
```

Recursion mental model

Define exactly what the function returns for one subtree. Then ask: **how does the parent combine the left result, current node, and right result?**

13

Binary Search Tree Patterns

Exploit the ordering property

BST property

For a conventional BST, values in the left subtree are smaller than the node and values in the right subtree are larger. Balanced height gives $O(\log n)$ search; a skewed tree can degrade to $O(n)$.

Search

```
Node cur = root;

while (cur != null) {
    if (cur.val == target) return cur;
    if (target < cur.val) cur = cur.left;
    else cur = cur.right;
}
return null;
```

Inorder insight

An inorder traversal of a BST visits values in sorted order. This is the bridge between tree problems and array-style ordering problems.

Validate BST — bounds pattern

```
boolean valid(Node node, long low, long high) {
    if (node == null) return true;

    if (node.val <= low || node.val >= high) return false;

    return valid(node.left, low, node.val) &&
           valid(node.right, node.val, high);
}
```

Common mistake

Checking only `node.left < node < node.right` is insufficient. A deep descendant must also respect all ancestor bounds.

14

Backtracking

Explore a decision tree and undo state

Core template

```
void backtrack(int start) {
    if (/* complete */) {
        // record answer
        return;
    }

    for (int i = start; i < n; i++) {
        // choose
        path.add(nums[i]);

        backtrack(i + 1);

        // undo
        path.remove(path.size() - 1);
    }
}
```

The three questions

Step	Question
Choose	What option am I taking at this level?
Explore	What does the recursive call need to know?
Undo	What state did I mutate that must be restored?

Common variants

- Subsets: choose / skip or loop from start.
- Combinations: enforce increasing indices to avoid duplicates/order duplicates.
- Permutations: choose any unused element; use boolean[] or a set of used indices.
- Grid word search: mark visited, recurse, then unmark.
- N-Queens: choose a row/column while maintaining attack constraints.

Pruning

Pruning means proving a branch cannot produce a valid/better answer and stopping early. Good pruning is often the difference between an unusable exponential search and an accepted solution.

Complexity

Backtracking is often exponential. State the branching factor and depth when explaining complexity rather than pretending every recursive algorithm is simply $O(n)$.

15

Greedy

Make a local choice that can be proven safe

What is greedy?

A greedy algorithm commits to the best-looking local choice without revisiting it. It is correct only when the problem has a property that makes that commitment safe.

Classic activity selection

For maximum number of non-overlapping intervals, sort by **ending time** and repeatedly choose the earliest finishing compatible interval.

```
Arrays.sort(intervals,
    Comparator.comparingInt(a -> a[1]));

int count = 0;
int end = Integer.MIN_VALUE;

for (int[] in : intervals) {
    if (in[0] >= end) {
        count++;
        end = in[1];
    }
}
```

How to recognize

- Scheduling / selecting maximum compatible activities.
- "Earliest finish" style structure.
- Need minimum resources and events can be ordered by endpoints.
- A local exchange argument seems possible.

Proof checklist

Proof idea	Question
Exchange argument	Can an optimal solution be transformed to use my greedy choice without becoming worse?
Stays ahead	After each step, is my greedy state at least as good as any competing strategy?
Invariant	What property remains true after each greedy choice?

Greedy warning

If you cannot explain why the local choice is safe, do not call the solution greedy just because it feels intuitive. Many greedy-looking problems actually require DP.

16

Graph Traversal Patterns

Represent relationships, then choose the traversal

Adjacency list

```
List<List<Integer>> graph = new ArrayList<>();

for (int i = 0; i < n; i++) {
    graph.add(new ArrayList<>());
}

graph.get(u).add(v);
// undirected:
// graph.get(v).add(u);
```

Connected components

Run DFS/BFS from every unvisited vertex. Each new start begins one component.

```
int components = 0;

for (int i = 0; i < n; i++) {
    if (!visited[i]) {
        components++;
        dfs(i);
    }
}
```

Cycle detection

The exact method depends on graph type. In an undirected graph, track the parent; in a directed graph, track recursion/three-state visitation.

Graph	Typical cycle idea
Undirected	DFS/BFS + parent, or Union-Find.
Directed	DFS color/state: unvisited, visiting, done; a back-edge to visiting means a cycle.

Topological sort

For a directed acyclic graph, Kahn's algorithm uses indegrees and a queue. Repeatedly remove zero-indegree nodes; if fewer than n nodes are processed, a cycle exists.

17

Shortest Path Patterns

Choose by edge-weight assumptions

Graph condition	First choice	Why
Unweighted / equal weight	BFS	First time reached = minimum edge count.
Non-negative weights	Dijkstra + PriorityQueue	Repeatedly finalize smallest tentative distance.
Edges have only 0/1 weights	0-1 BFS	Deque gives $O(V+E)$ style processing.
Negative edges	Bellman-Ford family	Can handle negative weights; detect negative cycles.

Dijkstra skeleton

```

PriorityQueue<long[]> pq =
    new PriorityQueue<>(Comparator.comparingLong(a -> a[0]));

long[] dist = new long[n];
Arrays.fill(dist, Long.MAX_VALUE);
dist[src] = 0;

pq.offer(new long[]{0, src});

while (!pq.isEmpty()) {
    long[] cur = pq.poll();
    long d = cur[0];
    int u = (int) cur[1];

    if (d != dist[u]) continue; // stale entry

    for (Edge e : graph.get(u)) {
        long nd = d + e.weight;

        if (nd < dist[e.to]) {
            dist[e.to] = nd;
            pq.offer(new long[]{nd, e.to});
        }
    }
}

```

Critical condition

Standard Dijkstra assumes **non-negative edge weights**. If negative edges exist, do not apply it blindly.

18

Dynamic Programming

Store answers to repeated subproblems

When to think DP

DP is appropriate when the problem has **overlapping subproblems** and an **optimal-substructure / recurrence**: the answer to a state can be built from answers to smaller states.

Four-step DP recipe

Step	Ask
1. State	What minimum information uniquely describes a subproblem?
2. Transition	How does this state come from smaller states?
3. Base case	What are the smallest states with known answers?
4. Order	In what order can all dependencies be computed?

1D DP example — climbing

```
int[] dp = new int[n + 1];
dp[0] = 1;
dp[1] = 1;

for (int i = 2; i <= n; i++) {
    dp[i] = dp[i - 1] + dp[i - 2];
}
```

Space optimization

If $dp[i]$ depends only on a fixed number of previous states, you may keep only those states instead of the full array.

Common DP families

Family	State idea
1D sequence	$dp[i]$ = best/ways up to i .
Knapsack	$dp[index][capacity]$ or compressed capacity.
Grid	$dp[r][c]$ = best/ways reaching cell.
String DP	$dp[i][j]$ = relationship between prefixes.
Subsequence	state often tracks index + previous choice/state.
Interval DP	$dp[l][r]$ = answer for interval.
Tree DP	subtree result returned upward.
Bitmask DP	state includes selected-set mask.

Do not memorize recurrences

Learn to derive the state from the question. If you cannot explain what $dp[i]$ means in one sentence, the DP is not ready.

19

Union-Find / DSU

Maintain connected components under repeated unions

Core operations

Disjoint Set Union maintains groups of elements. **find(x)** identifies the representative of x's component; **union(a,b)** merges two components.

```
int[] parent, size;

int find(int x) {
    if (parent[x] == x) return x;
    return parent[x] = find(parent[x]);
}

boolean union(int a, int b) {
    a = find(a);
    b = find(b);

    if (a == b) return false;

    if (size[a] < size[b]) {
        int t = a; a = b; b = t;
    }

    parent[b] = a;
    size[a] += size[b];
    return true;
}
```

When to use

- Dynamic connectivity.
- Detecting whether adding an edge creates a cycle.
- Kruskal's minimum spanning tree algorithm.
- Grouping equivalent items.

With path compression + union by size/rank, operations are effectively constant amortized time: $O(\alpha(n))$, where α is the inverse Ackermann function.

Core idea

A Trie stores characters along paths from a root. A word's path ends at a terminal marker. It is useful when the query is about prefixes rather than arbitrary substring membership.

```
class TrieNode {
    TrieNode[] next = new TrieNode[26];
    boolean end;
}

void insert(String s) {
    TrieNode cur = root;

    for (char c : s.toCharArray()) {
        int i = c - 'a';

        if (cur.next[i] == null)
            cur.next[i] = new TrieNode();

        cur = cur.next[i];
    }

    cur.end = true;
}
```

Operations

Operation	Typical cost	Use
insert(word)	O(L)	Add word.
search(word)	O(L)	Exact word lookup.
startsWith(prefix)	O(L)	Prefix lookup.

L is the length of the word/prefix. Memory can be large because nodes explicitly represent character transitions.

21

Bit Manipulation Patterns

Compact state and binary reasoning

Essential identities

Expression	Meaning
$x \& 1$	Checks whether x is odd (for non-negative integer reasoning).
$x \wedge x$	0
$x \wedge 0$	x
$x \& (x - 1)$	Clears the lowest set bit.
$x \& -x$	Isolates the lowest set bit (two's-complement integers).
$1 \ll k$	Bit mask with bit k set.

Common patterns

```
// set bit k
x = x | (1 << k);

// clear bit k
x = x & ~(1 << k);

// toggle bit k
x = x ^ (1 << k);

// test bit k
boolean set = (x & (1 << k)) != 0;
```

XOR pattern

XOR is useful when equal values cancel in pairs. For example, if every number appears twice except one, XORing all values leaves the unique value.

Caution

Bit tricks are powerful but should not replace a clearer solution when constraints do not require them. Understand signed integer shifts and overflow when using them.

22

Grid Patterns

Treat a matrix as a graph, geometry, or coordinate system

Direction vectors

```
int[] dr = {-1, 1, 0, 0};
int[] dc = {0, 0, -1, 1};
```

8 directions

```
int[] dr = {-1, -1, -1, 0, 0, 1, 1, 1};
int[] dc = {-1, 0, 1, -1, 1, -1, 0, 1};
```

Common grid families

Problem	Pattern
Island count / flood fill	DFS or BFS.
Shortest number of moves	BFS.
Number of paths	DP.
Connected regions with constraints	DFS/BFS + state.
Word search	Backtracking + visited.
Rotting / spread by minutes	Multi-source BFS.

Multi-source BFS

Initialize the queue with every starting source at distance 0. Process all sources simultaneously; the BFS layer number becomes elapsed time/distance.

23

Advanced Range Patterns

When ordinary loops are no longer enough

Difference between prefix, Fenwick and segment tree

Tool	Typical capability	When
Prefix sum	Static range sum / cumulative value.	Array does not change.
Difference array	Batch range updates then reconstruct.	Many offline range additions.
Fenwick tree	Point update + prefix/range aggregate.	Dynamic updates with simple associative operation.
Segment tree	Flexible range query + updates.	More complex dynamic range operations.

Fenwick tree core

```

void add(int i, int delta) {
    for (; i <= n; i += i & -i)
        bit[i] += delta;
}

int sum(int i) {
    int res = 0;
    for (; i > 0; i -= i & -i)
        res += bit[i];
    return res;
}

```

Fenwick trees use 1-based indexing in the standard implementation. Each operation is $O(\log n)$.

Segment tree recognition

Think segment tree when you need repeated range queries and updates and a simple prefix structure cannot handle the required operation efficiently.

24

Pattern Combinations

Most difficult problems use more than one idea

Combination	Typical example
Sort + Two Pointers	3Sum / pair relationships after ordering.
HashMap + Sliding Window	Longest substring with frequency constraints.
Prefix Sum + HashMap	Subarray sum equals K.
Binary Search + Greedy	Minimum capacity / speed / days where feasibility is monotonic.
Heap + HashMap	Top frequent elements / dynamic priorities.
Stack + Prefix/Suffix	Boundary/area calculations.
BFS + HashSet	State-space search with visited states.
Backtracking + Set	Avoid duplicate generated combinations.
DFS + DP memoization	Tree/graph state recursion with repeated states.
Sorting + Greedy	Interval scheduling / minimum resources.
DSU + Sorting	Kruskal MST.

Advanced recognition

Do not ask "Which single pattern is this?" Ask: **What is the main structure of the solution, and which tools support it?**

25

Practice Ladder

How to turn pattern knowledge into problem-solving ability

For every pattern

Level	Your task
0 — API drill	Write the Java template from memory.
1 — Tiny example	Trace 5–8 elements by hand.
2 — Simple	Solve without looking at a solution.
3 — Easy	Solve 2–4 variations with different edge cases.
4 — Medium	Explain the invariant before coding.
5 — LeetCode-level	Solve under time pressure; compare alternatives afterward.
6 — Revisit	Write down the exact reason your first attempt failed.

After each problem, record only five things

- **Trigger:** what clue told me the pattern?
- **State:** what variables did I maintain?
- **Invariant:** what was always true?
- **Complexity:** why is it $O(\dots)$ and space $O(\dots)$?
- **Failure:** what edge case or wrong assumption broke my first attempt?

Your problem notebook

Do not copy full solutions. Your notes should capture the **recognition rule + invariant + template modification + mistake**. That creates reusable pattern knowledge.

26

Master Pattern Decision Table

A fast lookup when you open a new DSA problem

If the problem says / implies...	Think first
Sorted array + pair / triplet	Two Pointers
Palindrome / compare from ends	Two Pointers
Remove duplicates / in-place compaction	Read-write Two Pointers
Contiguous subarray/substring	Sliding Window / Prefix Sum
Fixed length k range	Fixed Sliding Window
Exactly K distinct	atMost(K) – atMost(K-1)
Many range sums	Prefix Sum
Range updates then final values	Difference Array
Frequency / seen / complement	HashMap / HashSet
First/last occurrence	HashMap
Sorted / monotonic condition	Binary Search
Minimum feasible answer	Binary Search on Answer
Overlapping intervals	Sort + Interval scan
Next/previous greater/smaller	Monotonic Stack
Nested / most-recent unresolved	Stack
Shortest unweighted path	BFS
Level-by-level	BFS
Repeated min/max / Top K	PriorityQueue
Middle / cycle / reverse linked list	Fast/Slow or pointer manipulation
Tree property from children	DFS recursion
BST ordering	BST search / inorder
All combinations/permutations	Backtracking
Local choice can be proven safe	Greedy
Connectivity under unions	DSU
Prefix of words	Trie
Negative-free weighted shortest path	Dijkstra
Repeated overlapping subproblems	DP

27

Complexity Reference

The target you should state after every solution

Pattern	Typical time	Typical extra space
Two Pointers	$O(n)$ after sorting; sorting may be $O(n \log n)$	$O(1)$ to $O(n)$, depending on storage.
Sliding Window	$O(n)$ when each pointer moves monotonically	$O(k)$ / $O(\text{alphabet})$ / $O(1)$, depending on state.
Prefix Sum	$O(n)$ build + $O(1)$ query	$O(n)$
Hashing	$O(n)$ average for linear scan	$O(n)$
Binary Search	$O(\log n)$	$O(1)$ iterative.
Sort + scan	$O(n \log n)$	Depends on sort/representation.
Monotonic Stack	$O(n)$ amortized	$O(n)$
BFS/DFS	$O(V+E)$ graph; $O(RC)$ grid	$O(V)$ / $O(RC)$
Heap Top-K	$O(n \log k)$	$O(k)$
Backtracking	Often exponential	Recursion + state.
Greedy	Often $O(n \log n)$ if sorting is needed	Usually $O(1)$ – $O(n)$
DP	#states $\times$ transitions	#states, sometimes optimized.
DSU	$O(\alpha(n))$ amortized per operation	$O(n)$
Trie	$O(L)$ per operation	$O(\text{total characters})$
Dijkstra	$O((V+E) \log V)$ with binary heap style	$O(V+E)$

Complexity discipline

Always separate **algorithmic work** from **input preparation**. “ $O(n)$ ” is wrong if you first sort in $O(n \log n)$. Likewise, a HashMap-based $O(n)$ solution uses $O(n)$ additional memory.

28

Pattern Pitfalls

The mistakes that repeatedly appear in interviews

Pattern	Common mistake	Fix
Two pointers	Moving the wrong pointer without proving it can be discarded.	Write the sorted-order argument/invariant.
Sliding window	Using it when negative values destroy the needed monotonic behavior.	Consider prefix sum/hash or another technique.
Prefix sum	Off-by-one range formula.	Prefer $n+1$ prefix and $[l, r] \rightarrow p[r+1]-p[l]$.
Hashing	Storing the wrong occurrence.	Use <code>putIfAbsent</code> for first occurrence; overwrite for latest.
Binary search	Mixing inclusive/exclusive bounds.	Choose one template and preserve its invariant.
Intervals	Not deciding endpoint overlap semantics.	Clarify $<$ vs $\leq$ before coding.
Monotonic stack	Storing values when indices are needed.	Store indices when distance/bounds matter.
BFS	Marking visited too late and adding duplicates.	Usually mark when enqueueing.
Heap	Expecting iteration to be sorted.	Use repeated poll for sorted extraction.
Linked list	Losing next node before rewiring.	Save next first.
Tree recursion	Not defining return meaning.	Write "function(node) returns ..." before code.
Backtracking	Forgetting undo.	Every mutable choose needs a matching undo.
Greedy	No proof of safety.	Find exchange/stays-ahead/invariant or reconsider.
DP	State has too much/too little information.	Define the smallest sufficient state.

29

30-Day Pattern Study Plan

A practical sequence for limited daily study time

Days	Focus	Output
1–2	Recognition + Two Pointers	Template + 5 simple + 2 medium.
3–4	Sliding Window	Fixed + variable + exactly-K.
5	Prefix Sum / Difference	Range queries + subarray sum.
6–7	Hashing	Frequency, complement, grouping.
8–9	Binary Search	Exact, bounds, answer-space.
10	Sorting + Intervals	Merge, insert, scheduling.
11–12	Stack / Monotonic Stack	Parentheses + next greater + histogram.
13–14	Queue / BFS	Graph + grid + multi-source.
15	Heap / Top-K	Min/max + Top-K + K-way.
16–17	Linked List	Reverse, middle, cycle, merge.
18–19	Trees	DFS traversals + BFS + BST.
20–21	Backtracking	Subsets, combinations, permutations.
22	Greedy	Intervals + proof thinking.
23–25	Graphs	Components, cycles, topo, shortest path.
26–28	DP	1D, grid, knapsack/string families.
29	DSU + Trie + Bit	Recognition + core templates.
30	Mixed revision	10 problems: identify pattern before coding.

Daily routine for limited time

If you have only ~90 minutes: **20 min concept + 15 min template/tracing + 45 min problem solving + 10 min mistake log.** On very busy days, do the template + one problem rather than skipping the pattern completely.

Recognition

- I can identify 2–3 plausible patterns before coding.
- I can explain why one pattern fits better than the others.
- I can use constraints to reject slow approaches.

Implementation

- I can write the core template from memory.
- I know which Java structure supports the pattern.
- I can handle empty input, one element, duplicates and boundary cases.

Reasoning

- I can state the invariant in one sentence.
- I can explain why every pointer/stack/queue movement is safe.
- I can derive the recurrence for basic DP instead of memorizing it.

Complexity

- I can calculate time and auxiliary space.
- I distinguish amortized from worst-case where relevant.
- I include sorting/preprocessing cost.

Problem-solving maturity

- After a failed solution, I can identify the exact wrong assumption.
- I can recognize when two patterns should be combined.
- I can solve a familiar pattern without copying the previous solution.

The ultimate goal

When you open a new problem, you should naturally think: **input shape** → **constraints** → **pattern** → **invariant** → **Java structure** → **template** → **edge cases** → **complexity**. That is the skill this reference is designed to build.